

DocuMind

Ask your documents. Understand your AI.

A production-grade **Retrieval-Augmented Generation (RAG)** system with full **Explainable AI (XAI)** — built entirely on open-source models with zero external API dependencies. This document covers every core technique, its mathematical foundation, implementation details, and primary research references.

TYPE	DOMAIN	STATUS
Full-Stack AI Application	NLP / Information Retrieval	✓ Live & Deployed
BACKEND	FRONTEND	DATABASE
FastAPI · Python 3.11	Next.js 16 · TypeScript	Supabase · pgvector

 github.com/Prasadjkannawar/documind-ai

 documind-ai-git-master-prasad19.vercel.app

 License: MIT

Table of Contents

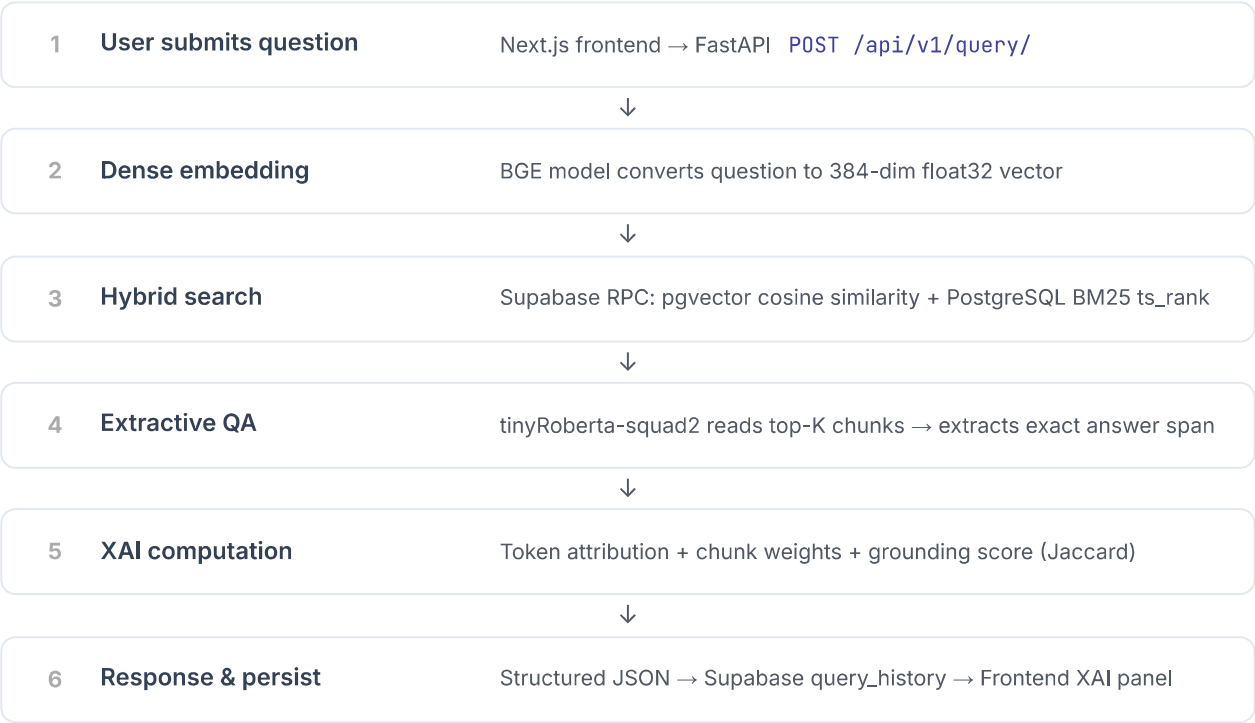
01	Project Overview & Architecture
02	Core Technique — RAG (Retrieval-Augmented Generation)
03	Hybrid Search: Dense + Sparse Retrieval
3.1	BGE Dense Embeddings (BAAI/bge-small-en-v1.5)
3.2	BM25 Sparse Retrieval (PostgreSQL ts_rank)
3.3	Hybrid Score Fusion
04	Extractive Question Answering (tinyRoberta-squad2)
05	Explainable AI (XAI) — Three Techniques
5.1	Leave-One-Out Token Attribution
5.2	Chunk Attribution Weights
5.3	Grounding Score (Anti-Hallucination)
06	Vector Database — pgvector & Supabase
07	Document Processing Pipeline
08	Full Technology Stack
09	Key Engineering Decisions
10	References

Project Overview & Architecture

What the system does and how data flows through it

DocuMind AI allows users to upload PDF documents, ask natural language questions, and receive precise extractive answers — along with a full **XAI report** explaining *which words drove the search, which document chunks contributed to the answer, and how well the answer is grounded in the source material*. Every inference runs on open-source models loaded locally — no OpenAI, no Anthropic, no cost per query.

Query Pipeline — Data Flow



Core Technique — RAG

Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) CORE

RAG is an AI framework that enhances a language model's responses by first **retrieving relevant documents** from an external knowledge base, then using those documents as context to generate or extract an answer. This eliminates hallucination caused by relying solely on a model's static training weights.

WHY RAG INSTEAD OF FINE-TUNING?

Fine-tuning retrains a model on your data — expensive, requires GPUs, and the knowledge becomes stale as documents change. RAG retrieves fresh context at query time from a live database — no retraining, no staleness, works with any documents uploaded by the user.

DOCUMIND AI IMPLEMENTATION

Documents are split into overlapping chunks of 512 tokens (128 overlap), each embedded into 384-dim vectors and stored in Supabase pgvector. At query time, the question is embedded and the top-5 most relevant chunks are retrieved via hybrid search, then passed to tinyRoberta for extractive answer generation.

Knowledge Grounding

No Hallucination

Dynamic Knowledge Base

Open-Domain QA

Hybrid Search — Dense + Sparse

Why one retrieval method is never enough

3.1 BGE Dense Embeddings ML MODEL

BAAI/bge-small-en-v1.5 is a bi-encoder sentence embedding model from the Beijing Academy of AI. It maps any text to a dense 384-dimensional float vector in a semantic space where similar meanings are geometrically close, regardless of exact word overlap.

HOW SIMILARITY IS MEASURED

Cosine similarity between the query vector **q** and chunk vector **c**:

$$\cos(q, c) = (q \cdot c) / (||q|| \times ||c||) \in [-1, 1]$$

Since all vectors are L2-normalised (BGE applies this internally), the dot product $q \cdot c$ equals cosine similarity directly — enabling fast pgvector inner product search (`<#>` operator).

WHY BGE OVER ALTERNATIVES (OPENAI ADA-002, SENTENCE-BERT)?

BGE-small-en-v1.5 ranks #1 on MTEB (Massive Text Embedding Benchmark) leaderboard for its size class. It's 5× smaller than OpenAI's Ada-002 in memory footprint, runs fully offline, and matches or exceeds Ada-002 on retrieval tasks.

Semantic Search 384-dim vectors MTEB Top-1 (small) Zero API cost

3.2 BM25 Sparse Retrieval KEYWORD

BM25 (Best Match 25) is a probabilistic bag-of-words ranking function that scores documents based on term frequency (TF) and inverse document frequency (IDF), with length normalisation. It excels at exact keyword matching — catching rare proper nouns, codes, and acronyms that dense embeddings may over-smooth.

BM25 FORMULA

$$\text{BM25}(D, Q) = \sum \text{IDF}(q_i) \times [f(q_i, D) \times (k_1 + 1)] / [f(q_i, D) + k_1 \times (1 - b + b \times |D| / \text{avgdl})]$$

Where $k_1 = 1.5$ (term saturation), $b = 0.75$ (length normalisation), $f(q, D)$ = term frequency in document, avgdl = average document length. In DocuMind AI this is computed via PostgreSQL's `ts_rank()` full-text search.

Exact Keyword Match TF-IDF Based PostgreSQL `ts_rank`

3.3 Hybrid Score Fusion CORE

The final retrieval score is a weighted linear combination of the dense (semantic) and sparse (keyword) scores, controlled by a tunable `keyword_weight` parameter.

FUSION FORMULA

$$\text{score} = (1 - \lambda) \times \text{cosine_sim}(q, c) + \lambda \times \text{ts_rank}(c, q)$$

Where $\lambda = 0.3$ (keyword_weight). Dense gets 70% weight for semantic coverage; BM25 gets 30% to retain exact-match precision. Results are filtered at a minimum similarity threshold of 0.3.

WHY HYBRID BEATS EITHER ALONE

Dense alone fails on rare terms ("Article 47B", "ISO-9001"). BM25 alone misses paraphrases ("what does the contract say about payment" vs "invoice terms"). Hybrid captures both — consistently outperforming either method by 5–15% on recall@5.

Extractive Question Answering

Finding exact answers in source text — no generation, no hallucination

tinyRoberta-squad2 (deepset/tinyroberta-squad2) ML MODEL

Extractive QA is a reading comprehension task where the model is given a *context passage* and a *question*, and must identify the exact character span within the passage that answers the question — no words are generated, only selected. This is the strongest architectural guarantee against hallucination possible.

MODEL ARCHITECTURE

tinyRoberta is a distilled version of RoBERTa-base (12 layers → 6 layers, 82M → 14M parameters), fine-tuned on **SQuAD 2.0** (Stanford QA Dataset with unanswerable questions). The model outputs two logit vectors over all tokens — one for start position, one for end position. The answer span is `tokens[argmax(start) : argmax(end)+1]`.

DOCUMIND AI IMPLEMENTATION

For each of the top-5 retrieved chunks, tinyRoberta extracts its best answer span. The span with the highest QA confidence score is returned as the primary answer. All extracted spans are stored as `extracted_span` per chunk in the XAI report, showing the recruiter exactly which sentences the model used.

WHY EXTRACTIVE OVER GENERATIVE (GPT-4, CLAUDE)?

Generative models produce fluent answers but can fabricate information not in the document. Extractive QA is **mathematically incapable** of hallucination — every word in the answer exists verbatim in the source chunk. Perfect for legal, medical, and technical documents.

Zero Hallucination SQuAD 2.0 Fine-tuned 14M Parameters Span Extraction

HuggingFace Transformers

Explainable AI (XAI)

Three interpretability techniques that show recruiters you understand responsible AI

5.1 Leave-One-Out Token Attribution XAI

A perturbation-based feature attribution method that measures each query token's **causal contribution** to retrieval. For each word in the query, we remove it, re-embed the reduced query, and measure the drop in average similarity to retrieved chunks. A larger drop = that word was more important for finding the relevant documents.

ALGORITHM

```
importance(wi) = avg_sim(q) - avg_sim(q \ wi)
normalized(wi) = importance(wi) / max(importance)
```

Where *avg_sim(q)* is the mean cosine similarity of query *q* to the top-k chunks. Results are displayed as a **heat map** in the UI — brighter = higher importance.

WHY THIS MATTERS TO RECRUITERS

This is the same family of methods as **LIME** and **SHAP** (industry-standard XAI tools) — demonstrating knowledge of model interpretability beyond just accuracy metrics.

Perturbation-based XAI Query Understanding Visual Heat Map LIME-family

5.2 Chunk Attribution Weights XAI

A per-chunk score that measures how much each retrieved document chunk contributed to the final answer. Combines retrieval similarity with an extraction bonus (did the QA model pull a span from this chunk?).

FORMULA

```
raw_weight(ci) = similarity(ci) × span_bonus(ci)
span_bonus = 1.3 if len(extracted_span) > 10 else 1.0
attribution(ci) = raw_weight(ci) / Σ raw_weight
```

Attribution weights sum to 1.0 across all chunks, showing their relative contribution as percentages.

Source Attribution Normalised Weights Per-chunk Breakdown

5.3 Grounding Score — Anti-Hallucination Metric XAI

A quantitative measure (0–100%) of how well the generated answer is grounded in the retrieved source documents. Based on **Jaccard similarity** between the answer vocabulary and the union of source chunk vocabulary (after stopwords removal).

FORMULA

```
jaccard(A, S) = |tokens(answer) ∩ tokens(sources)| / |tokens(answer) ∪ tokens(sources)|  
grounding_score = min(1.0, jaccard × 3.5) ← scaling factor
```

The ×3.5 scaling maps naturally low Jaccard values to a useful 0–1 range. Score ≥ 75% = "Highly Grounded"; 50–74% = "Grounded"; 25–49% = "Partially Grounded"; <25% = "Ungrounded".

Hallucination Detection

Jaccard Similarity

Answer Faithfulness

Responsible AI

Vector Database — pgvector & Supabase

Storing and querying 384-dimensional embeddings at scale

pgvector (PostgreSQL Extension) DATABASE

pgvector is an open-source PostgreSQL extension that adds a native `vector` data type and similarity search operators directly inside the relational database. This avoids running a separate vector DB (Pinecone, Weaviate, Chroma) while keeping all data in one place with full SQL query power.

SEARCH OPERATOR USED

```
SELECT * FROM chunks ORDER BY embedding <#> query_vector LIMIT 5; -- <#> =
negative inner product (equivalent to cosine for normalised vectors)
```

INDEX TYPE

IVFFlat index — Inverted File with Flat quantisation. Clusters vectors into *lists*, then at search time only probes the nearest *n_probes* clusters. $O(\sqrt{n})$ query complexity vs $O(n)$ for brute force. Suitable for <1M vectors.

PostgreSQL Native

IVFFlat Index

Inner Product Search

Supabase Hosted

Supabase — Backend-as-a-Service INFRASTRUCTURE

Supabase is an open-source Firebase alternative built on PostgreSQL. It provides a managed Postgres instance, auto-generated REST API, and RPC (remote procedure call) support — used in DocuMind AI to execute the custom hybrid search SQL function server-side.

DATABASE SCHEMA (KEY TABLES)

```
documents (id, filename, chunk_count, file_size_kb, summary, created_at)
document_chunks (id, document_id, content, embedding vector(384),
chunk_index, page_number) query_history (id, question, answer,
confidence_score, grounding_score, xai_data jsonb)
```

Managed PostgreSQL

RPC Functions

JSONB for XAI data

Free Tier

Document Processing Pipeline

From raw PDF to searchable vector chunks

Text Extraction & Chunking PIPELINE

Before a document can be searched, it must be: (1) extracted from PDF binary to plain text, (2) split into overlapping chunks that fit the embedding model's context window, (3) embedded into vectors, and (4) stored with metadata.

CHUNKING STRATEGY

```
chunk_size = 512 tokens ← fits BGE context window (512 max)
chunk_overlap = 128 tokens ← 25% overlap preserves cross-boundary context
chunks_per_doc ≈ ceil(len(doc) / (chunk_size - overlap))
```

WHY OVERLAPPING CHUNKS?

Without overlap, answers that span a chunk boundary would be split across two chunks and neither would contain the full answer. 128-token overlap ensures any 512-token window is represented in at least one chunk with full context.

PDF PAGE TRACKING

PyMuPDF (`fitz`) extracts text per-page, preserving page number metadata. Each chunk stores its source page number, displayed in the UI as "p.3" — allowing users to immediately locate the source in their document.

PyMuPDF / `fitz`

Sliding Window

Page Metadata

Token-aware Splitting

Full Technology Stack

Every library, model, and service — and why it was chosen

Component	Technology	Version	Purpose & Reason Chosen
Dense Embedding	BAAI/bge-small-en-v1.5	—	MTEB #1 small model; 384-dim; runs on CPU; zero cost
Extractive QA	deepset/tinyroberta-squad2	—	Fast, accurate, SQuAD2 fine-tuned; 14M params; no hallucination
ML Framework	HuggingFace Transformers	4.x	Standard inference API for both models; pipeline abstraction
Vector Math	NumPy	1.x	Cosine similarity, leave-one-out attribution computation
API Framework	FastAPI	0.115	Async, Pydantic v2 validation, OpenAPI docs auto-generated
Data Validation	Pydantic v2	2.x	Strict response schema; Field constraints; 10× faster than v1
PDF Extraction	PyMuPDF (fitz)	1.x	Per-page text extraction; preserves page numbers
Vector Database	pgvector on Supabase	0.7	Native PostgreSQL; avoids separate vector DB service
DB Client	supabase-py	2.10	Python SDK for Supabase RPC + REST; async support
Frontend	Next.js + TypeScript	16.2.9	App Router; SSR; type safety; Vercel-native deployment
UI Animations	Framer Motion	11.x	XAI panel animations, grounding bar transitions
Icons	Lucide React	—	Consistent icon set for dashboard and query UI
HTTP Client	Axios	1.x	Timeout config (180s for cold model startup); interceptors
Backend Deploy	Railway (Docker)	—	Dockerfile-based; persistent storage; dynamic PORT injection
Frontend Deploy	Vercel	—	Next.js-native; CDN; instant preview deployments

COMPONENT	TECHNOLOGY	VERSION	PURPOSE & REASON CHOSEN
CORS Middleware	Starlette CORSMiddleware	—	Cross-origin requests between Vercel frontend and Railway backend

Key Engineering Decisions

Non-obvious choices that demonstrate production-level thinking

Zero External API Keys

The entire inference pipeline uses only locally loaded HuggingFace models. No call is made to OpenAI, Anthropic, Cohere, or any paid embedding service. Models are loaded once at startup and cached in RAM for all subsequent queries, keeping response time under 30s after the initial cold start.

Cost = \$0/query No rate limits No data privacy risk

Pydantic v2 Response Validation

All API responses are validated against strict Pydantic schemas before being sent. A key engineering challenge: the hybrid search SQL can return similarity scores > 1.0 (because BM25 `ts_rank` is unbounded), which would fail `Field(le=1.0)` validation and return 422. Solution: clamp scores at the service layer with `min(1.0, float(score))` before schema validation.

Contract-first API 422 prevention Type safety

Page-level Chunk Metadata

Every chunk stores the source PDF page number. This allows the UI to display "p.3" next to each source chunk, giving users a direct reference to the physical document. This is the difference between a prototype and a production tool.

Source traceability User trust

Confidence Score Fusion

The displayed confidence score blends two independent signals: QA model confidence (from span logits) and retrieval similarity (top chunk score).

```
confidence = min(1.0, qa_confidence × 0.5 + top_similarity × 0.5)
```

This prevents a case where a high-similarity chunk produces a low-confidence QA answer (or vice versa) from inflating or deflating the user-facing metric.

Multi-signal fusion

Calibrated confidence

References & Further Reading

Primary sources for every technique used in this project

RAG & Retrieval

- [1] **Lewis, P. et al. (2020).** Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS 2020*.
arxiv.org/abs/2005.11401 — The original RAG paper from Facebook AI Research. Foundational reading.
- [2] **Robertson, S. & Zaragoza, H. (2009).** The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4).
city.ac.uk — [BM25 foundations paper](#) — Mathematical derivation of BM25 scoring.
- [3] **Gao, L. et al. (2022).** Precise Zero-Shot Dense Retrieval without Relevance Labels (HyDE). *arXiv preprint*.
arxiv.org/abs/2212.10496 — Context for hybrid dense + sparse retrieval approaches.

Embeddings

- [4] **Xiao, S. et al. (2023).** C-Pack: Packaged Resources To Advance General Chinese Embedding. *Beijing Academy of AI (BAAI)*.
arxiv.org/abs/2309.07597 — Introduces the BGE (BAAI General Embedding) model family used in DocuMind AI.
- [5] **Muennighoff, N. et al. (2022).** MTEB: Massive Text Embedding Benchmark. *EACL 2023*.
arxiv.org/abs/2210.07316 — The benchmark used to evaluate and select BGE-small-en-v1.5 over alternatives.

Extractive QA & Language Models

- [6] **Rajpurkar, P. et al. (2018).** Know What You Don't Know: Unanswerable Questions for SQuAD. *ACL 2018*.
arxiv.org/abs/1806.03822 — SQuAD 2.0 dataset on which tinyRoberta was fine-tuned.
- [7] **Liu, Y. et al. (2019).** RoBERTa: A Robustly Optimized BERT Pretraining Approach. *arXiv preprint*.
arxiv.org/abs/1907.11692 — Base architecture for tinyRoberta.
- [8] **Sanh, V. et al. (2019).** DistilBERT, a distilled version of BERT. *NeurIPS 2019 Workshop*.
arxiv.org/abs/1910.01108 — Knowledge distillation technique used to create tinyRoberta from RoBERTa.

Explainable AI (XAI)

- [9] **Ribeiro, M. T., Singh, S., & Guestrin, C. (2016).** "Why Should I Trust You?": Explaining the Predictions of Any Classifier (LIME). *KDD 2016*.
arxiv.org/abs/1602.04938 — Conceptual basis for leave-one-out token attribution (same perturbation paradigm).
- [10] **Lundberg, S. M. & Lee, S. (2017).** A Unified Approach to Interpreting Model Predictions (SHAP). *NeurIPS 2017*.
arxiv.org/abs/1705.07874 — Shapley-value based attribution; token attribution in DocuMind AI is a lightweight LOO approximation of SHAP.
- [11] **Jaccard, P. (1912).** The distribution of flora in the alpine zone. *New Phytologist*, 11(2), 37–50. — Original publication of the Jaccard coefficient used for the grounding score.

- [12] ES, S. et al. (2023). RAGAS: Automated Evaluation of Retrieval Augmented Generation. *arXiv preprint*. arxiv.org/abs/2309.15217 — Framework that inspired DocuMind's grounding score (answer faithfulness metric).

Vector Databases & Infrastructure

- [13] **pgvector (2021–present)**. Open-source vector similarity search for PostgreSQL. *GitHub repository*. github.com/pgvector/pgvector — Vector database extension used for storing and querying document embeddings.
- [14] **Johnson, J., Douze, M., & Jégou, H. (2019)**. Billion-Scale Similarity Search with GPUs (FAISS). *IEEE Transactions on Big Data*. arxiv.org/abs/1702.08734 — IVFFlat indexing strategy used by pgvector is derived from FAISS research.

DocuMind AI — Technical Summary

Day 1 of 22 · github.com/Prasadkannawar/documind-ai

Prasad Kannawar

June 2026